BHU-DRISHTI · ENGINEERING DESIGN · OCTOBER 2026

# AEQE Engineering

How the Anytime Error-bounded Query Engine is built: processes, threads, data paths, caches, failure handling and measured performance. This document covers the implementation; the project guide covers the statistics.

**6,937**
lines of Python in 24 modules (`engine/aeqe`)

**21**
HTTP endpoints (FastAPI)

**6**
data-source adapters behind one interface

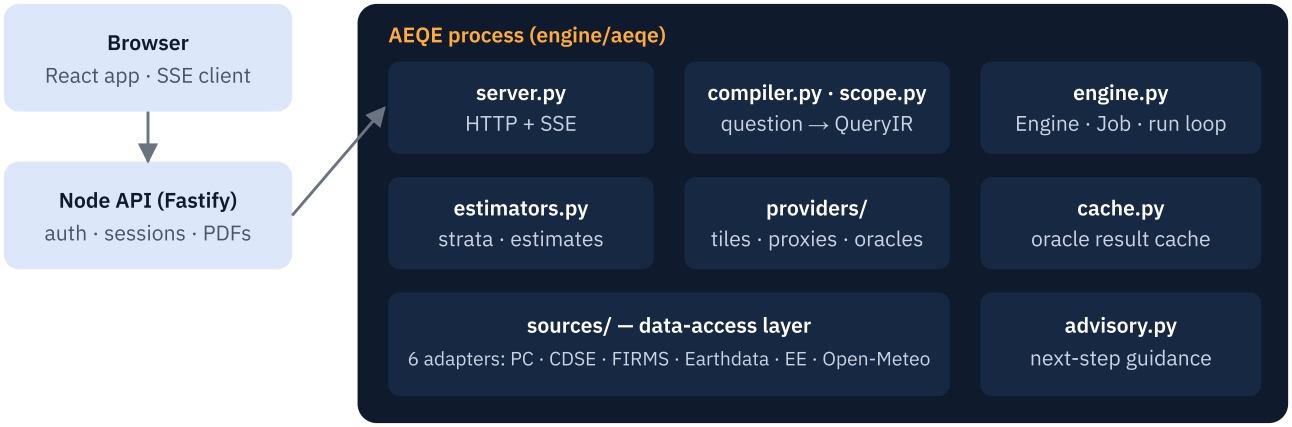
**67**
automated engine tests, all passing

Team: Bikash Kumar Sah · Pramugdha Gupta · Aditi. Facts in this document are taken from the code at the time of writing; measured numbers come from runs on the team's development machine in India.

1 · ARCHITECTURE

# Where AEQE sits and what it is made of

AEQE is a Python service (FastAPI + Uvicorn) that turns one question into a running “job”. In the deployed container it runs next to the Node API, bound to `127.0.0.1:8000`, so only the API can reach it.




| Module | Lines | Responsibility |
|---|---|---|
| `providers/live.py` | 1,122 | India districts and custom areas on live data: tiling, static layers, proxies, the four oracles, supporting evidence, provenance. |
| `engine.py` | 932 | `Engine` and `Job`: validation, planning, the sampling loop, census and find-all paths, stopping, results, traces, persistence. |
| `estimators.py` | 692 | Stratification, Neyman allocation, stratified and cross-fitted difference estimators, Clopper–Pearson bounds, recall audit. |
| `providers/simulated.py` | 559 | Two demo districts with known ground truth, for tests and benchmarks. |
| `compiler.py` · `scope.py` · `ir.py` | 642 | Rule compiler (optional LLM compiler), out-of-scope replies, the typed `QueryIR`. |
| `server.py` | 396 | 21 HTTP endpoints, the Server-Sent Events stream, the archive fallback after restarts. |
| `sources/*` | 1,667 | One adapter per provider, a shared HTTP helper, status ladder, secret redaction, TTL cache, registry. |
| `cache.py` · `advisory.py` · `composite.py` | 327 | Oracle result cache (SQLite), recovery guidance, routing between demo and live providers. |

# From `POST /queries` to the final event

**1 Compile** `compile_question()` → `QueryIR`; off-topic → HTTP 422 with suggestions

**2 Submit** `Engine.submit()` creates a `Job` (12-hex id) and starts a daemon thread

**3 Validate** blocking `Checks`; a failure emits `plan` + `error` and ends

**4 Prepare** tile universe, proxies, threshold; status lines streamed to the page

**5 Sample** batches through the oracle pool; a `progress` event per batch

**6 Finish** result, advisory, trace and archive written; `result` event

## The run loop (engine.py, simplified)

```
while True:
    # stop checks: user stop, time / tile budget, replay length
    requested = _batch_for_time(job, ir, t_start)   # shrinks near the deadline
    alloc = state.allocate(requested, residual_sd)  # Neyman, with a floor
    idxs  = state.take(alloc)
# random order inside strata
    results = pool.map(_call_oracle, idxs)          # cache-first, parallel
    state.record(...)
    eligible = state.n >= 60
    conf_k = spent_confidence(conf, looks)          # alpha spending
    est = state.best(X, conf_k)
# difference or stratified
    job.emit({"type": "progress", ...})
    if eligible and precision_met(est):
break   # first qualifying look
```

## Three execution paths

- **Sampling** (area share, total, count): the loop on the left.
- **Census** ( `active_fire` ): `provider.census_values()` counts every FIRMS detection per tile; no oracle calls, no interval.
- **Find-all**: searches tiles in proxy order and audits the remainder ( `FrontierAudit` ) until a Clopper–Pearson bound certifies the recall target.

## Budgets

Each query carries `budget_seconds` and `budget_tiles` : 120 s / 1,000 tiles by default, **900 s** for live districts, 300 s / 5,000 for find-all, 200,000 tiles for census. The time budget includes preparation, but at least one batch always runs.

## Events streamed to the page

| Event | When | Payload |
|---|---|---|
| `status` | during preparation | human-readable stage, elapsed seconds (via a thread-local status callback) |
| `plan` | after validation and tiling | checks, sensor choice, oracle description, strata, stop rule |
| `progress` | after every batch | n, population, oracle calls, cache hits, elapsed, estimate with interval, allocation, the batch's tiles |
| `evidence` | when context arrives | rainfall, MODIS flood, forecasts (never mixed into the number) |
| `result` · `error` · `end` | at the end | full result object; or the error; `end` closes the stream |

# The three objects everything passes around

### `QueryIR` (ir.py, Pydantic)

```
task          area_fraction | total | count | find_all | inspect
target        flooded_cropland | new_builtup | brick_kiln
              | active_fire | burned_cropland
region_id     e.g. "in-assam-barpeta"
start, end    date window (+ baseline_start/end for change)
sensor        auto | sentinel-1 | sentinel-2 | fusion
precision     ±pp (share) · ±% (total/count) · recall % (find-all)
confidence    0.5 < c < 1 (default 0.95)
budget_seconds, budget_tiles
seed          7 (fixed → reproducible sample order)
point, radius_km   inspect only
question      the original text
```

The compiler is the only place that reads free text. Everything after it works on this typed, validated object, so the engine is deterministic for a given IR and seed.

### `TileUniverse` (providers/base.py)

```
tile_ids      list[str]     "g0.01:90.9646:26.5102"
bboxes        (N, 4)        lon/lat boxes, clipped to boundary
area_km2      (N,)
base_km2      (N,)          denominator, e.g. cropland km²
features      (N, p)        cheap covariates for the proxy model
score0        (N,)          initial score → strata
extra         threshold_db, tile_deg, scene counts …
```

All arrays are aligned by index; the estimators only ever see indices into this population. Tile size is the smallest of 0.01°, 0.02°, 0.025°, 0.05°, 0.1° that keeps a district under **8,000 tiles**.

### `Job` (engine.py)

- `status`: queued → running → done | stopped | failed.
- `events`: an append-only list. Each event gets a sequence number, so a client can reconnect with `?since=n` and miss nothing.
- `_cond`: a `threading.Condition`; `emit()` appends and notifies, `wait_events()` blocks up to 1 s.
- `universe`, `strata`, `state`: the population, its strata and the running sample (`SampleState`).
- `trace`: every batch, allocation, estimate and input scene, written as JSON at the end.
- `replay_of` / `replay_n`: re-run the same sample sequence to reproduce or audit a past answer.

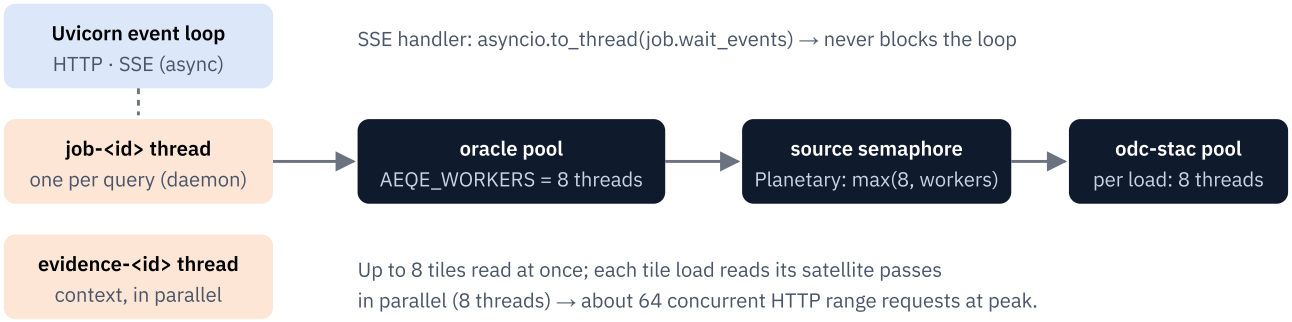
### The result object (what the API stores and the PDFs read)

`estimate` (value, lo, hi, halfwidth, method, interval method, km² totals) · `stop_reason` · `n`, `population`, `oracle_calls`, `cache_hits`, `elapsed_s` · `sensor`, `oracle`, `scene_key` · `caveats` · `missing_data_tiles` · `supporting_evidence` · `inputs_summary` · `advisory` · `answer` (the sentence shown to users) · `query` (the IR). The Node API keeps a whitelisted snapshot of these fields per session.

4 · CONCURRENCY MODEL

# Threads, pools and the limits between them

Live work is network-bound (reading satellite imagery over HTTP), so AEQE uses threads, not processes. Image reads in GDAL/rasterio release Python's GIL while waiting on the network.




## Why three layers of limits

- **Oracle pool** (`ThreadPoolExecutor`, shared by all jobs): how many tiles are analysed at once.
- **Source semaphore** (`BoundedSemaphore` per adapter): protects each provider from overload and throttling, across jobs and evidence threads. Planetary Computer gets one slot per oracle worker.
- **Load pool** (`AEQE_LOAD_THREADS`): within one tile, the 6–12 satellite passes are fetched concurrently instead of one after another.

## Thread-safety rules in the code

- Job events go through one `Condition`; readers never touch the list without the lock.
- Provider caches (items, universes, static layers, land cover) are guarded by one provider lock.
- The SQLite result cache uses one connection with `check_same_thread=False`, WAL mode and an internal lock.
- The status callback is **thread-local**, so concurrent jobs never stream each other's progress messages.

**Known limitation:** AEQE is a single process. Jobs live in an in-memory dictionary and are never evicted, so memory grows with the number of queries until the engine restarts. Finished jobs are archived to disk, so a restart loses nothing finished, but horizontal scaling (several engine instances) would need a shared job store.

5 · DATA-ACCESS LAYER

# Six providers behind one interface

Every provider is a subclass of `Source` (sources/base.py). The engine never calls a provider's HTTP API directly; it calls adapter methods that return arrays or records with provenance attached.

### What every adapter gets for free

- **`http_request()`**: bounded retries with backoff, timeouts, one credential refresh on 401, and errors turned into `SourceError(kind)`: `timeout`, `network`, `auth`, `forbidden`, `quota`, `rate_limited`, `not_found`, `bad_request`, `server`, `unavailable`.
- **`redact()`**: strips keys, tokens and signed URLs from every message and from library log records before they reach logs or users.
- **Provenance**: each answer lists the scenes and products it used (id, date, resolution, asset).
- **TTL cache** for small metadata calls, and a **status ladder** shown in the app:

`not_configured → configured → authenticated → discovery_verified → retrieval_verified → e2e_verified · or blocked (with a precise reason)`

### Reading satellite images (Planetary Computer)

- STAC search through `pystac-client`; items are signed with short-lived SAS tokens.
- Pixels are read with `odc.stac.load()` onto an EPSG:4326 grid for the tile's box, so only the needed bytes of each Cloud-Optimised GeoTIFF are fetched (HTTP range requests).
- **Retry policy** (4 attempts): an expired signature triggers one re-sign; transient errors back off $1.5 \cdot 2^{k}$ s. The last attempt tolerates failing assets, which become "no data" instead of losing the tile.
- **GDAL tuning**, set at import: `GDAL_DISABLE_READDIR_ON_OPEN=EMPTY_DIR`, `CPL_VSIL_CURL_ALLOWED_EXTENSIONS=.tif`, merged byte ranges, HTTP/2 multiplexing, `VSI_CACHE`.

| Adapter | Used for | Role in an answer |
|---|---|---|
| Planetary Computer | Sentinel-1 RTC, Sentinel-2 L2A, ESA WorldCover, Copernicus DEM | **Primary**: proxies and every oracle read |
| NASA FIRMS | VIIRS and MODIS fire detections | Census for fires; proxy for burned cropland |
| Copernicus Data Space | Sentinel-1 catalogue, Statistical API | Cross-check and freshness; processing-unit budget enforced ( `CDSE_PU_BUDGET` ) |
| NASA Earthdata | GPM IMERG rainfall, MODIS NRT flood | Supporting evidence only |
| Google Earth Engine | AlphaEarth embeddings | Optional extra proxy features |
| Open-Meteo | Rain and river-discharge forecasts | Supporting evidence, labelled as forecast |

# What runs before sampling, and what one tile costs

## Preparing a district (flood example)

1. **Static layers** (first run per district only): read ESA WorldCover, tile the boundary, compute cropland/water/built fractions per tile. Saved as `data/static/<region>_<tile>.npz`.
2. **Event overview**: one coarse (~200 m) Sentinel-1 read of the whole district for the question's window.
3. **Regional threshold**: Otsu on the event histogram, clamped to −24…−14 dB, else −18 dB; one threshold for every tile.
4. **Dry-season reference** (Feb–Apr): median of up to 34 passes at the same coarse grid; **cached on disk**.
5. **Proxy features**: new-water fraction, current water, × cropland area, cropland, permanent water; plus AlphaEarth PCs if Earth Engine is configured. Tiles with < 0.02 km² cropland are dropped from the population.

| Oracle | Reads per tile | Rule (deterministic, no trained model) | Grid |
|---|---|---|---|
| `s1_flood_change` | event passes (min), up to 12 reference passes (median, cached), WorldCover, DEM | flooded = water now (VV < threshold), not water in reference, on cropland, not permanent water, slope ≤ 5° | ~20 m |
| `s2_builtup_change` | Jan–Mar stacks for two years (8 clearest scenes, cloud-masked) | persistent built-up: NDBI > 0.05 and never green; new = built now and not in baseline | ~20 m |
| `s2_dnbr_burn` | pre and post Sentinel-2 composites, WorldCover | dNBR > 0.27, post NBR < 0.1, dark char surface, on cropland; FIRMS reported alongside | ~20 m |
| `firms_detections` | FIRMS detections (one area request) | count per tile (census) | points |

## Oracle calls are cache-first

`_call_oracle()` builds a key from **tile id + scene key + oracle name + model version + parameters hash**. On a hit it returns instantly and reports the seconds saved. On a miss it runs the oracle, retries once, and on a second failure records the tile as *no data* (value 0, reason kept) rather than failing the job.

## Why the scene key matters

The scene key lists the exact satellite passes used. When a new pass arrives for the same window, the key changes, the old results miss, and nothing stale is reused. Bumping an oracle's `model_version` invalidates its cache the same way.

# Six caches, each with its own key and lifetime

| Store | Where | Key | Lifetime / invalidation |
|---|---|---|---|
| Oracle results | `data/cache.sqlite` (WAL) | tile, scene key, oracle, version, params hash | permanent; new scenes or versions miss automatically; `POST /cache/clear` |
| Dry-season reference | `data/refcache/xx/<sha1>.npy` (float32) | sorted pass ids + box + resolution | permanent; a new pass changes the key. ~40 KB per tile |
| Static layers | `data/static/*.npz` | region + tile size | permanent; land cover does not change |
| STAC item lists | memory (256 entries) | window + region (reference: season only) | until restart; oldest dropped first |
| Universes, land cover per tile | memory | region, target, task, dates · tile box | until restart (land cover cleared at 2,000 tiles) |
| Source metadata | `data/sources.sqlite` (TTL cache) | source + request | time-to-live per call |

### What survives a restart

- **Finished jobs**: on completion, `_archive()` writes `data/jobs/<id>.json` (summary + evidence map) atomically (temp file, then rename).
- **Traces**: `data/traces/<id>.json`, the full audit trail of batches and inputs.
- After a restart, `/queries/{id}`, `/evidence`, `/events` and `/trace` fall back to these files. Ids are checked to be alphanumeric before any path is built.

### What does not

- **Running jobs** are lost on restart. The Node API detects the engine's 404 and marks the session *failed: interrupted* instead of leaving it "running".
- In-memory caches rebuild on demand.
- On a free host without a persistent disk, the whole `data/` folder is temporary: the engine still works, just with cold caches after each restart.

**Design rule:** every cache is keyed by its *inputs*, never by time or by the question. That is why a cache can be kept forever and shared between users without ever serving a stale answer.

8 · FAILURE HANDLING

# What happens when things go wrong

| Failure | Behaviour |
|---|---|
| Question out of scope (weather, AQI, cricket…) | `OutOfScopeError` → HTTP 422 with a category and suggested questions; no job is created. |
| Data or region invalid | A blocking `Check` fails (e.g. task/target mismatch, point outside region, no scenes): `plan` + `error` events, job status `failed`. |
| One tile read fails | Retried once; then recorded as *no data* with the reason. The estimate continues on the other tiles. |
| Many tiles unreadable (cloud, gaps) | Stop reason becomes `insufficient_evidence`: the answer says no reliable estimate exists and shows only a partial figure. |
| Expired signed URL (403) | Items are re-signed once and the read is retried. |
| Provider throttling or outage | `SourceError` with its kind; retryable kinds back off and retry. The last error is recorded per source and shown on the Data sources page; a failing live check marks the source *blocked* with the reason. |
| Supporting evidence slow | The answer is sent after at most **5 s** of waiting; late evidence is added to the result, the archive and the stream (which stays open up to 2 minutes for it). |
| Time budget about to run out | `_batch_for_time()` shrinks the next batch using the 75th-percentile tile latency, so one slow batch cannot overrun the budget by much. |
| User presses stop | The loop checks `stop_requested` before each batch; the interval at that point is still valid (anytime). |
| Engine restarts | Finished jobs are served from the archive; running jobs are reported as interrupted by the API. |
| Any unexpected exception | `_run_safe()` catches it, marks the job failed, emits an `error` event, and still archives what exists. |

**Security notes.** Credentials are read from the environment only and never logged: `redact()` also rewrites log records of odc, rasterio, pystac and urllib3. The engine binds to `127.0.0.1` inside the container, so the browser can only reach it through the authenticated Node API. Archive and trace lookups validate ids before touching the filesystem.

# Measured latency, and what was done about it

Live flood question for Barpeta (Assam, 2,123 tiles), run on the development machine in India. Every satellite read crosses to Microsoft's West Europe region, so network latency dominates.

| Stage | Before | After | Change |
|---|---|---|---|
| One tile, 12 reference passes | 11.4–12.5 s | 2.9–5.2 s | passes read in parallel (odc-stac pool) + GDAL HTTP settings |
| District dry-season reference (34 passes) | 40 s | 8 s · 0.1 s cached | parallel reads; on-disk reference cache |
| Preparation total | 51 s | 16 s | both of the above |
| Per batch (18–24 tiles) | 55–100 s | ~45 s | parallel pass reads |
| Batches run | 4 | 3 | stop at the first look that meets the precision (valid with alpha spending) |
| Waiting for context after the answer | up to 90 s | ≤ 5 s | answer first, evidence streamed later |
| **Fresh question, end to end** | **358 s** | **146 s** | ≈ 2.5× faster |
| Same district, new dates | — | 125 s | reference from cache |

### Where the remaining time goes

- ~16 s preparation, then ~45 s per batch of fresh tiles.
- Each tile still needs about 20 COG range reads (event passes, reference, land cover, elevation) across continents.
- At least 60 tiles are needed before the interval is trusted (the minimum sample for variance estimates), so a fresh flood question needs about 3 batches.

### Next levers (not yet done)

- Run the engine close to the data (a West-Europe host): network latency per read drops several-fold.
- Pre-warm references and overviews nightly for the 735 districts.
- A smaller first batch, so a provisional answer arrives sooner.
- Cache land cover and elevation per tile on disk, like the reference.

Simulated benchmark (no network): 3.7–13× fewer oracle calls than processing every tile, with 95% intervals covering the exhaustive answer 97–100% of the time on flood questions (engine/bench/RESULTS.md).

# HTTP API and configuration

| Endpoint | Purpose |
|---|---|
| `GET /health` | liveness, provider, live-data flag, interval mode, compiler |
| `GET /regions` · `/regions/{id}` · `POST` · `DELETE` | 735 districts + custom areas (GeoJSON/KML uploads); search and boundaries |
| `GET /regions/{id}/freshness` | latest available scene per product (used by monitors) |
| `GET /sources` · `POST /sources/check` | source status ladder; bounded live smoke test |
| `POST /compile` | question → IR, assumptions, warnings, suggested region; 422 when out of scope |
| `POST /queries` · `GET /queries` · `GET /queries/{id}` | start a job; list; summary (archive fallback) |
| `GET /queries/{id}/events?since=n` | Server-Sent Events stream, resumable by sequence number |
| `POST /queries/{id}/stop` | request a stop at the next look |
| `GET /queries/{id}/universe` · `/evidence` · `/trace` | tile map with proxy ranks; sampled-tile map; audit trail |
| `POST /queries/{id}/replay` · `/exhaustive` | reproduce a run; run the oracle on every tile (validation) |
| `GET /cache/stats` · `POST /cache/clear` | oracle cache size and hit statistics; reset |

| Setting | Default | Effect |
|---|---|---|
| `AEQE_PROVIDER` | auto | auto (demo + live) · simulated · live |
| `AEQE_WORKERS` · `AEQE_BATCH` | 8 · 24 | parallel tile analyses; tiles per batch |
| `AEQE_LOAD_THREADS` | 8 | parallel pass reads inside one tile |
| `AEQE_SEQUENTIAL` | spending | anytime-valid intervals; `fixed` for the faster, simulation-only mode |
| `AEQE_DATA_DIR` · `AEQE_CACHE_PATH` | ./data | where caches, archives and traces live |
| `AEQE_LLM` (+ model, key, base URL) | off | optional LLM compiler; the rule compiler is the default |
| `AEQE_SIM_LATENCY` | 0.12 s | simulated oracle delay for demo districts |
| `GDAL_HTTP_*` · `CDSE_PU_BUDGET` | 4 retries · 3,000 | raster read retries and timeout; Copernicus processing budget |
| Provider keys | — | `NASA_FIRMS_MAP_KEY`, `CDSE_CLIENT_ID/SECRET`, `EARTHDATA_TOKEN`, `GOOGLE_*`: each unlocks a source |

# Testing, and honest limitations

## 67 automated engine tests (offline)

- **Estimators**: strata construction, Neyman allocation, Clopper–Pearson bounds, recall audit.
- **Compiler**: parametrised phrasings → task/target; out-of-scope categories; region mentions.
- **Adapters**: HTTP mocked; status ladder, redaction, error classification.
- **Oracle rules**: flood, built-up and burn masks on synthetic arrays.
- **Engine paths**: census, flaky oracle → no-data tiles, insufficient evidence, early stop at the first qualifying look, archive after restart, reference cache reuse and invalidation, every module compiles.
- Benchmarks (`engine/bench`): 100 seeds per scenario against known ground truth.

CI (GitHub Actions) runs these, the backend tests and the frontend build on every push, then starts the full container and checks it end to end.

## Known engineering limitations

- **Single process**; jobs held in memory and never evicted. Needs a job store for scale-out and an eviction policy for long uptimes.
- **No live exhaustive validation yet**: the coverage numbers are from simulated districts. The `/exhaustive` endpoint exists to close this gap.
- **Oracles are rule-based** and not validated against field data; burned cropland is marked experimental.
- **Network-bound**: most time is spent on long-distance image reads; placement near the data matters more than CPU.
- **Shared free hardware** (2 vCPU): several concurrent live jobs slow each other; the API limits each user to 2 parallel runs.
- **Cold caches on free hosts** without a persistent disk.

## Summary for reviewers

AEQE is a single-process, thread-based Python service that compiles a question into a typed query, prepares a tiled population of a district from cached and live layers, and streams a sampling estimate whose interval stays valid at every look. Its engineering choices — cache keys derived from inputs, cache-first oracle calls, bounded concurrency per provider, atomic archives, and failure modes that degrade to *no data* rather than wrong numbers — are what make a statistical idea usable on free, slow, unreliable satellite data.